Reviewer Pack — Width-5 ABPs for Parity Require Depth Ω(log n)

Entry aceb7cb67219 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-19 06:05:52 UTC

Width-5 ABPs for Parity Require Depth Ω(log n)

Entry ID: `aceb7cb67219` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-19 06:05:52 UTC

1. Conjecture

Field A (mathematical branch): ALGEBRAIC_BRANCHING_PROGRAMS (ABPs) **Field B** (complexity object): NC¹_CIRCUIT_COMPLEXITY

Statement:

> Any width-5 ABP computing the parity function on n bits must have depth Ω(log n).

Rationale (proposer's reasoning):

> Barrington's theorem shows NC¹ functions can be computed by width-5 ABPs, but parity (in $AC^0 \subseteq NC^1$) may require deeper ABPs. This conjecture explores depth-width trade-offs for parity, leveraging known AC^0 depth lower bounds to infer ABP depth constraints.

Taxonomy category: `BARRINGTON_ALG` (status at proposal time: partially_alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `b3834844ad60e97e`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> default: >=80% seeds must support, no counterexample

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.95	UNCERTAIN	SAFE
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.95	UNCERTAIN	SAFE
KARP_LIPTON	SAFE	0.95	UNCERTAIN	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_abps():
        if n == 1:
            yield [0]
            return
        for left in generate_abps():
            for right in generate_abps():
                if len(left) + len(right) <= 4:
                    yield left + right

    def evaluate(abp):
        if not abp:
            return 0
        elif len(abp) == 1:
            return abp[0]
        else:
            mid = len(abp) // 2
            return (evaluate(abp[:mid]) + evaluate(abp[mid:])) % 2

    def is_parity_function(abp):
        for i in range(1 << n):
            if evaluate([i] * n) != parity(i):
                return False
        return True

    def parity(x):
        x ^= x >> 1
        x ^= x >> 2
        x ^= x >> 4
        x ^= x >> 8
        x ^= x >> 16
        return x & 1

    n = random.randint(5, 40)
    abps = list(generate_abps())
    tested_count = min(len(abps), 30)  # Ensure at least 30 instances are tested
    depth_sum = 0

    for _ in range(tested_count):
```

```
        abp = random.choice(abps)
        if is_parity_function(abp):
            depth_sum += len(abp)

    avg_depth = depth_sum / tested_count if tested_count > 0 else 0
    conjecture_holds = avg_depth >= math.log2(n)
    counterexample = "" if conjecture_holds else f"Average depth {avg_depth} < log2({n}) = {math.l

    return {
        "metric_name": "average_abp_depth",
        "metric_value": avg_depth,
        "instances_tested": tested_count,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i + 1 for i in range(5, 30)]

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_depth = sum(r["metric_value"] for r in results) / len(results)
    std_dev = math.sqrt(sum((r["metric_value"] - mean_depth)**2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_depth} std={std_dev} support_fraction={support_fracti
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_depth} std={std_dev} support_fraction={support_fracti
    else:
        first_failing_seed = next(seed for seed, r in zip(seeds, results) if not r["conjecture_hol
        print(f"RESULT: FALSIFIED counterexample=\"Average depth too low\" first_failing_seed={fir
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_67e018d3.py", line 81, in <module>
    trial_result = run_trial(seed)
                   ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_67e018d3.py", line 54, in run_trial
    abps = list(generate_abps())
           ^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_67e018d3.py", line 25, in generate_abps
    for left in generate_abps():
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_67e018d3.py", line 25, in generate_abps
```

```
    for left in generate_abps():
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_67e018d3.py", line 25, in generate_abps
    for left in generate_abps():
  [Previous line repeated 995 more times]
RecursionError: maximum recursion depth exceeded
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> Test crashed due to infinite recursion, preventing data collection | next: Fix generate_abps() recursion and re-run with seed sampling

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	118146
2	propose	ollama_remote	qwen3:8b	0	0	109419
3	preregistration	ollama_remote	qwen3:8b	0	0	27619
4	novelty	ollama_remote	qwen3:8b	0	0	23958
5	novelty	ollama_remote	qwen3:8b	0	0	27814
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12327
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10385
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8601
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8883
10	judge	ollama_remote	qwen3:8b	0	0	42187

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 389340 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/aceb7cb67219.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/aceb7cb67219.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/aceb7cb67219.tar.gz` (if generated)